TALLER DE PREPARACIÓN PARA EL PARCIAL

Preparar al estudiante para el parcial mediante la resolución de un problema empresarial aplicando programación orientada a objetos, UML básico, estructuras de control, colecciones, manejo de excepciones, hilos, documentación con Javadoc y generación de un archivo JAR. El estudiante deberá analizar el problema, diseñar una solución, implementarla en Java, probarla y presentar evidencias de su funcionamiento.

1. Caso guiado:

Sistema de gestión de pedidos de una cafetería universitaria

Una cafetería universitaria necesita un sistema que registre pedidos de estudiantes. Cada pedido contiene el nombre del cliente, una lista de productos y un estado. El valor total se calcula a partir de los productos registrados. El sistema debe permitir registrar varios pedidos, validar información y simular la preparación de varios pedidos al mismo tiempo, como si diferentes empleados atendieran pedidos concurrentemente.

1.1 Análisis del problema

- Clase Producto: representa un artículo de la cafetería.
- Atributos de Producto: nombre y precio.
- Clase Pedido: representa el pedido realizado por un cliente.
- Atributos de Pedido: cliente, productos y estado.
- El total no se almacena como atributo; se calcula mediante calcularTotal().
- Colección: List<Producto>, implementada mediante ArrayList.
- Excepciones: validar nombre vacío y precio negativo.
- Hilos: PreparacionPedido implementa Runnable.
- Javadoc: documentar clases, constructores y métodos principales.

1.2 Clase Producto

```
import java.util.Objects;

/**
 * Representa un producto vendido en la cafetería.
 */
public class Producto {
   private String nombre;
   private double precio;

   /**
    * Crea un producto con nombre y precio válidos.
    *
    * @param nombre nombre del producto
    * @param precio precio del producto
```

```
     * @throws IllegalArgumentException si el nombre está vacío o el precio es negativo
     */
    public Producto(String nombre, double precio) {
        if (nombre == null || nombre.trim().isEmpty()) {
            throw new IllegalArgumentException("El nombre no puede estar vacío.");
        }
        if (precio < 0) {
            throw new IllegalArgumentException("El precio no puede ser negativo.");
        }

        this.nombre = nombre;
        this.precio = precio;
    }

    public String getNombre() {
        return nombre;
    }

    public double getPrecio() {
        return precio;
    }
}
```

1.3 Clase Pedido

```
import java.util.ArrayList;
import java.util.List;

/**
 * Gestiona un pedido realizado por un cliente.
 */
public class Pedido {
    private String cliente;
    private List<Producto> productos;
    private String estado;

    /**
     * Crea un pedido para un cliente.
     *
     * @param cliente nombre del cliente
     * @throws IllegalArgumentException si el cliente está vacío
     */
    public Pedido(String cliente) {
        if (cliente == null || cliente.trim().isEmpty()) {
```

```
            throw new IllegalArgumentException("El cliente no puede estar vacío.");
        }

        this.cliente = cliente;
        this.productos = new ArrayList<>();
        this.estado = "Registrado";
    }

    /**
     * Agrega un producto al pedido.
     *
     * @param producto producto que se agregará
     */
    public void agregarProducto(Producto producto) {
        if (producto == null) {
            throw new IllegalArgumentException("El producto no puede ser nulo.");
        }
        productos.add(producto);
    }

    /**
     * Calcula el valor total del pedido.
     *
     * @return total del pedido
     */
    public double calcularTotal() {
        double total = 0;

        for (Producto p : productos) {
            total += p.getPrecio();
        }

        return total;
    }

    /**
     * Cambia el estado del pedido.
     *
     * @param nuevoEstado nuevo estado del pedido
     */
    public void cambiarEstado(String nuevoEstado) {
        if (nuevoEstado == null || nuevoEstado.trim().isEmpty()) {
```

```
            throw new IllegalArgumentException("El estado no puede estar vacío.");
        }
        estado = nuevoEstado;
    }

    /**
     * Muestra un resumen del pedido.
     */
    public void mostrarResumen() {
        System.out.println("Cliente: " + cliente);
        System.out.println("Estado: " + estado);
        System.out.println("Total: $" + calcularTotal());
    }

    public String getCliente() {
        return cliente;
    }

    public String getEstado() {
        return estado;
    }
}
```

1.4 Clase PreparacionPedido

```
/**
 * Simula la preparación de un pedido utilizando un hilo.
 */
public class PreparacionPedido implements Runnable {
    private Pedido pedido;

    public PreparacionPedido(Pedido pedido) {
        this.pedido = pedido;
    }

    @Override
    public void run() {
        try {
            pedido.cambiarEstado("En preparación");
            System.out.println("Preparando pedido de " + pedido.getCliente());

            Thread.sleep(2000);

            pedido.cambiarEstado("Entregado");
```

```
            System.out.println("Pedido entregado a " + pedido.getCliente());

        } catch (InterruptedException e) {
            pedido.cambiarEstado("Interrumpido");
            Thread.currentThread().interrupt();
            System.out.println("La preparación fue interrumpida.");
        }
    }
}
```

1.5 Clase Main: menú y concurrencia

```
import java.util.Scanner;

/**
 * Clase principal del sistema.
 */
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Ingrese el nombre del cliente: ");
            String cliente = sc.nextLine();

            Pedido pedido = new Pedido(cliente);

            int opcion;

            do {
                System.out.println("\n===== CAFETERÍA =====");
                System.out.println("1. Agregar café");
                System.out.println("2. Agregar empanada");
                System.out.println("3. Mostrar resumen");
                System.out.println("4. Procesar pedido");
                System.out.println("5. Salir");
                System.out.print("Seleccione una opción: ");

                opcion = Integer.parseInt(sc.nextLine());

                switch (opcion) {
                    case 1:
                        pedido.agregarProducto(new Producto("Café", 2500));
                        System.out.println("Café agregado.");
```

```
                break;

            case 2:
                pedido.agregarProducto(new Producto("Empanada", 3000));
                System.out.println("Empanada agregada.");
                break;

            case 3:
                pedido.mostrarResumen();
                break;

            case 4:
                Thread hilo = new Thread(new PreparacionPedido(pedido));
                hilo.start();
                hilo.join();
                break;

            case 5:
                System.out.println("Fin del programa.");
                break;

            default:
                System.out.println("Opción no válida.");
        }

      } while (opcion != 5);

    } catch (NumberFormatException e) {
      System.out.println("Debe ingresar un número válido.");
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      System.out.println("El proceso fue interrumpido.");
    } catch (IllegalArgumentException e) {
      System.out.println("Error: " + e.getMessage());
    } finally {
      sc.close();
    }
  }
}
```

1.6 Conceptos que deben identificarse

Concepto	Dónde aparece	Qué debe comprender el estudiante
POO	Producto, Pedido, PreparacionPedido	Clases, objetos y responsabilidades
Encapsulación	Atributos private	Protección de datos mediante métodos
Colecciones	List y ArrayList	Almacenar y recorrer objetos
If	Validaciones	Tomar decisiones
Switch	Menú	Seleccionar opciones
For	calcularTotal()	Recorrer una colección
while/do-while	Menú	Repetir procesos
Excepciones	try-catch / throw	Lanzar y controlar errores
Hilos	Runnable / Thread	Procesamiento concurrente
Javadoc	/** */	Documentación técnica

2. Práctica autónoma: Sistema de turnos para soporte técnico

Una empresa de soporte técnico necesita organizar la atención de solicitudes realizadas por usuarios. Cada solicitud tiene número de ticket, nombre del usuario, tipo de problema, prioridad y estado. El sistema deberá registrar solicitudes, almacenarlas en una colección, validarlas, mostrarlas y procesarlas de manera concurrente utilizando hilos.

2.1 Actividades a desarrollar

1. Leer el caso y subrayar los sustantivos que puedan convertirse en clases, atributos u objetos.
2. Identificar las clases Solicitud, TecnicoSoporte y ProcesadorSolicitud y justificar su responsabilidad.
3. Diseñar un diagrama de clases UML básico antes de programar.
4. Crear la clase Solicitud con atributos privados, constructor, métodos get/set y métodos de comportamiento.
5. Crear una colección List<Solicitud> para almacenar varias solicitudes.
6. Implementar un menú utilizando switch y un ciclo do-while o while.
7. Validar campos vacíos y prioridades incorrectas utilizando if y excepciones.
8. Crear una clase que implemente Runnable para procesar cada solicitud.
9. Crear y ejecutar al menos dos hilos utilizando start().
10. Agregar Javadoc a cada clase, constructor y método principal.
11. Ejecutar el programa y capturar evidencia de la salida.
12. Generar la documentación Javadoc.
13. Generar un archivo JAR ejecutable como actividad de cierre.

2.2 Requisitos mínimos

- Al menos tres clases relacionadas.
- Atributos privados y métodos públicos.
- Constructor para las clases principales.
- Una colección para almacenar objetos.
- Uso de if y switch.
- Uso de for y while/do-while.
- Uso de try-catch y al menos una excepción de validación.
- Al menos una clase que implemente Runnable o extienda Thread.
- Ejecución de mínimo dos hilos mediante start().
- Javadoc en clases, constructores y métodos principales.
- Programa compilable y ejecutable sin errores.
- Archivo JAR ejecutable.

2.3 Reto adicional: start(), run() y join()

14. Explicar la diferencia entre iniciar un hilo con start() y llamar directamente a run().
15. Modificar el programa para que el hilo principal espere a que terminen los procesadores mediante join().
16. Explicar qué sucede si no se utiliza join() en un proceso que requiere esperar a los hilos.

3. Diagrama de clases UML esperado

El diagrama debe incluir nombre de las clases, atributos, métodos, tipos de datos, visibilidad y relaciones principales. Como mínimo, se espera una estructura similar a:

Solicitud

- numeroTicket : int
- usuario : String
- tipoProblema : String
- prioridad : int
- estado : String

+ Solicitud(...)
+ cambiarEstado(...)
+ mostrarInformacion()

TecnicoSoporte ---> Solicitud
ProcesadorSolicitud ---> Solicitud

4. Evidencias de entrega

- Documento breve con análisis del caso.
- Diagrama de clases UML.
- Código fuente completo del proyecto Java.

- Capturas de pantalla de la ejecución.
- Documentación Javadoc generada.
- Archivo JAR ejecutable.
- Respuestas a las preguntas de reflexión.

5. Preguntas de reflexión y análisis

1. ¿Por qué los hilos permiten mejorar la simulación del proceso de atención de solicitudes?
2. ¿Qué diferencia existe entre extender Thread e implementar Runnable?
3. ¿Qué diferencia existe entre start() y run()?
4. ¿Para qué sirve join() en un programa con hilos?
5. ¿Qué ventajas ofrece Javadoc para el mantenimiento del software?
6. ¿Qué errores podrían presentarse si no se controlan las excepciones?
7. ¿Cuál es la diferencia entre lanzar una excepción con throw y capturarla con try-catch?
8. ¿Cómo se evidencia la programación orientada a objetos en la solución desarrollada?
9. ¿Por qué es conveniente utilizar una colección de objetos en lugar de declarar muchas variables independientes?
10. ¿Qué estructura de control utilizaría para construir un menú y por qué?